

DevOps Introduction

Lucas Bonanni

What is DevOps?

What is DevOps?

DevOps surgió de la colisión de dos tendencias importantes relacionadas.

La primera se denominó “infraestructura ágil” u “operaciones ágiles”; Surgió de la aplicación de enfoques Agile y Lean al trabajo de operaciones.

El segundo es una comprensión ampliada del valor de la colaboración entre el personal de desarrollo y operaciones a lo largo de todas las etapas del ciclo de vida de desarrollo al crear y operar un servicio, y cuán importantes se han vuelto las operaciones en nuestro mundo cada vez más orientado a los servicios.

What is DevOps?

DevOps es la práctica en la que los ingenieros de operaciones y desarrollo participan juntos en todo el ciclo de vida del servicio, desde el diseño, pasando por todo el proceso de desarrollo, hasta la implementación y el soporte de producción.

What is DevOps?

DevOps también se caracteriza por el hecho de que el personal de operaciones utiliza muchas de las mismas técnicas que los desarrolladores para el trabajo de sus sistemas, y el personal de desarrollo crea aplicaciones diseñadas para funcionar bien en un entorno operativo.

What is DevOps?

Las técnicas de desarrollo que las operaciones aprenden a utilizar en un entorno de DevOps van desde el uso de control de versiones hasta el desarrollo de la infraestructura como código (incluyendo pruebas) y la participación en un proceso de desarrollo ágil. Las preocupaciones operativas integradas en los sistemas incluyen la observabilidad y la resiliencia. Ambos tienden a utilizar la integración y entrega continuas como el medio para iterar en una solución.

What is DevOps?

“DevOps” pretende ser lo más inclusivo posible. No diferencia entre diferentes subdisciplinas de administradores de sistemas: “Ops” es un término general para ingenieros de sistemas, administradores de sistemas, personal de operaciones, ingenieros de lanzamiento, administradores de bases de datos, ingenieros de redes, profesionales de seguridad y varias otras subdisciplinas y títulos de trabajo. “Dev” se utiliza como abreviatura para desarrolladores front-end y back-end en particular, pero en la práctica es aún más amplio y significa “todas las personas involucradas en el desarrollo del producto”, que pueden incluir Producto, Control de Calidad y otras disciplinas.

CAMS

CAMS

Culture Automation Measurement Sharing



DevOps Foundation®

BLUEPRINT

With DevOps, people across the IT organization, working together, enable fast flow, feedback and continuous improvement of planned work into production, while achieving quality, stability, reliability, availability, security and team satisfaction.

CALMS Values

C: Culture - emphasizes shared vision collaboration, communication, learning and continuous improvement.

A: Automation - CI/CD toolchains, and infrastructure-as-code enable automation, consistency, velocity and fast recovery.

L: Lean - Maximize customer value while minimizing waste and improving flow.

M: Measurement - Value-driven metrics for people, process and technology support trust and performance improvement.

S: Sharing - Leaders and teams share ideas, and skills, improve communication, collaboration and performance.

The Three Ways

1st Way: Continuous Flow
2nd Way: Feedback
3rd Way: Continuous Improvement
(Experimenting and Learning)

Organization

Cross-function teams focus on business goals. Ops work with Dev, supporting each other to improve flow towards production and monitor results.



Principles & Practices

Frequent small releases using continuous integration, testing, delivery, deployments and monitoring reduce lead time, costs and risks.

Related Frameworks

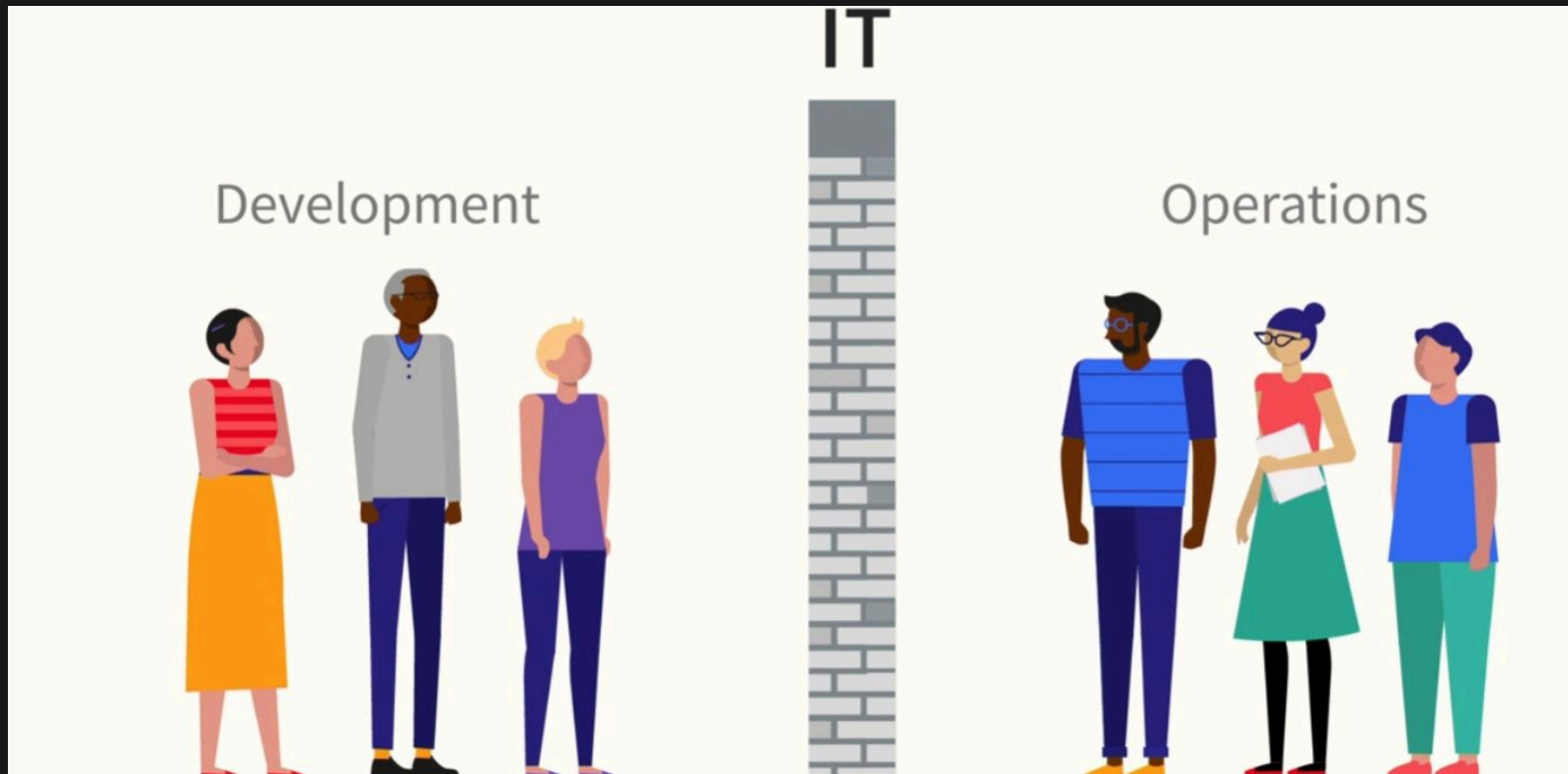
Agile: Lean Development
ITSM: Processes
Lean: Reduce Waste
Value Stream Management: End-to-End

Benefits

Improved release cadence, velocity, throughput, efficiency and stability, quality, security and team satisfaction.

Culture

La cultura en DevOps va más allá de las comodidades físicas y se centra en los comportamientos y la comprensión mutua entre los equipos.



Automation

La automatización es un acelerador que permite controlar sistemas y aplicaciones.

People over Process over Tools

1. People
2. Process
3. Tools

Measurement

La medición es esencial para una aproximación racional a los sistemas.

Métricas clave: MTTR, cycle time, costos, ingresos, satisfacción del empleado.

Measure Key Metrics **across** the Organization



Mean Time to Recovery



Cycle Time



Costs



Revenue



Employee Satisfaction

Sharing

Compartir ideas y problemas es fundamental para la colaboración y la mejora continua.

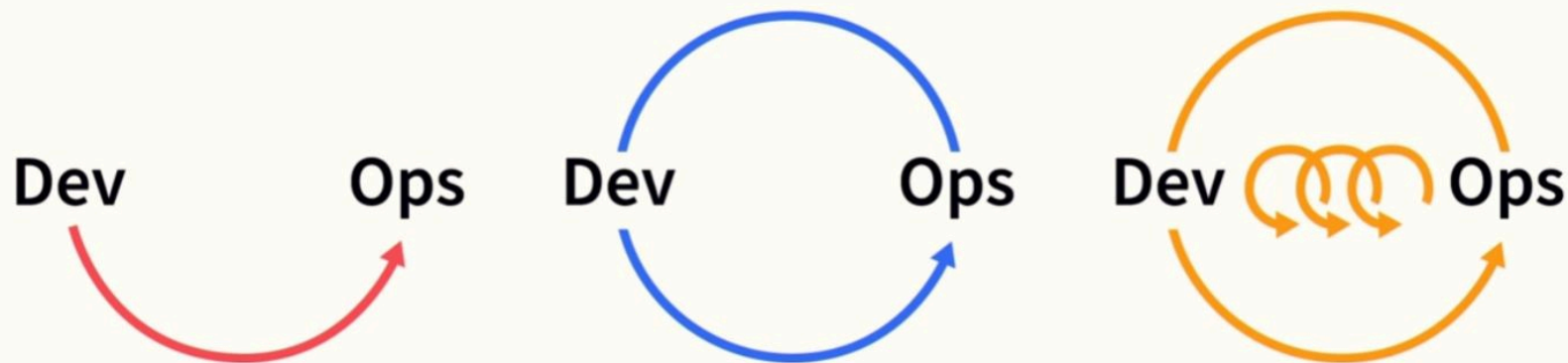
Kaizen ()¹: In a business context, continuous improvement.

1. [wiki/Kaizen](https://es.wikipedia.org/wiki/Kaizen) ↩

The Three Ways

The Three Ways

- System Thinking
- Feedback loops
- Continuous experimentation and learning



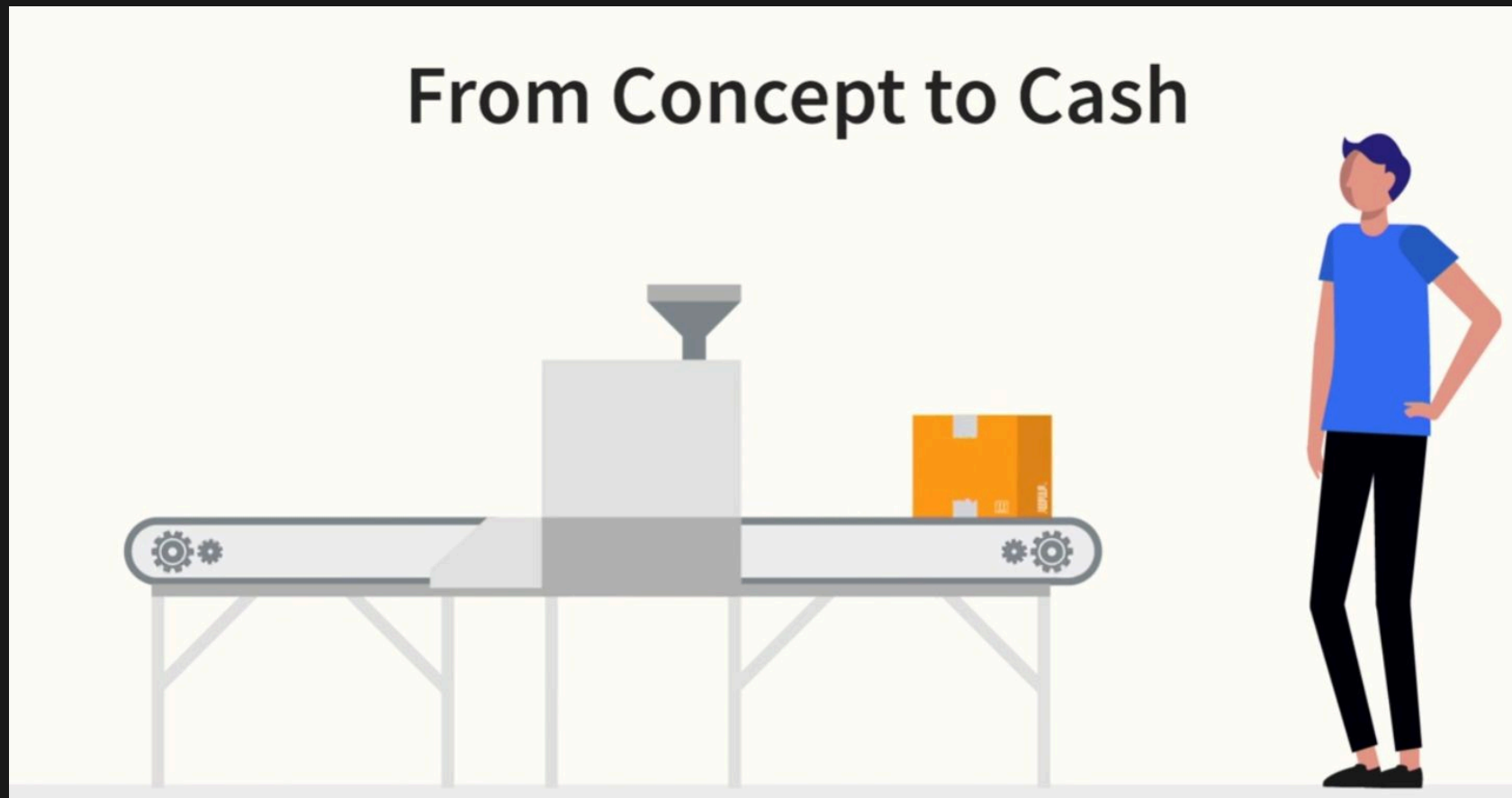
System Thinking

Comprender el sistema en su totalidad, desde el concepto hasta la entrega, en lugar de optimizar partes individuales a expensas del resultado general.



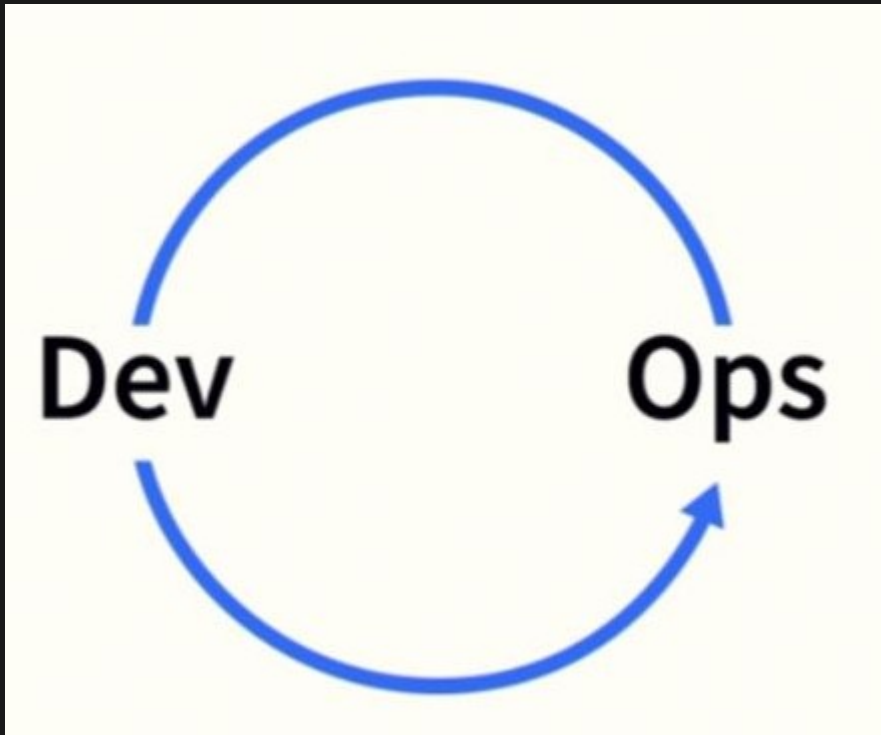
From Concept to Cash

El flujo completo desde la conceptualización de una idea hasta su entrega y monetización.

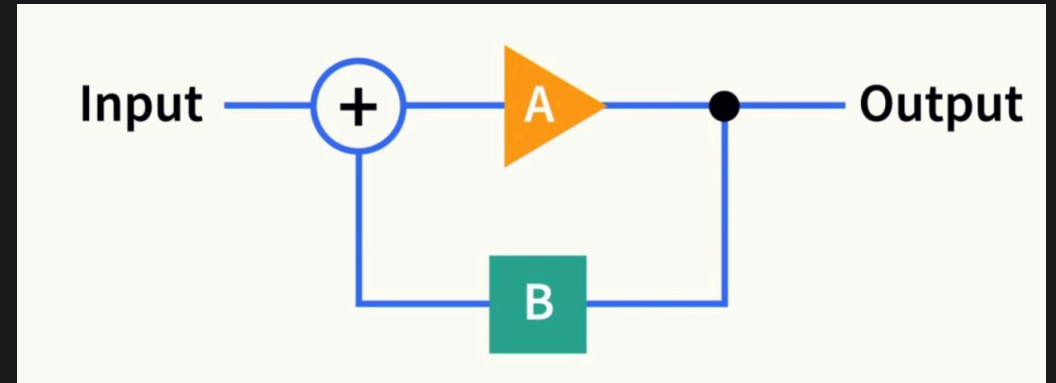


Amplify Feedback Loops

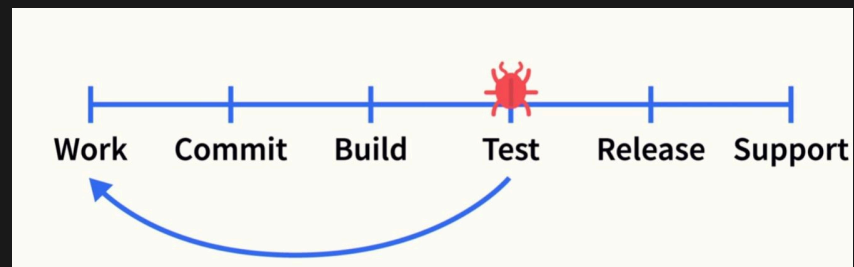
Crear, acortar y amplificar los bucles de retroalimentación entre los diferentes equipos y partes de la organización.



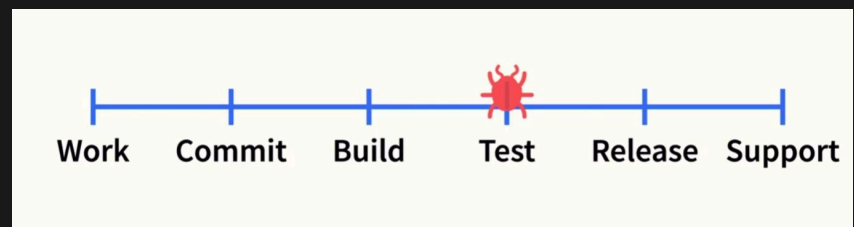
1



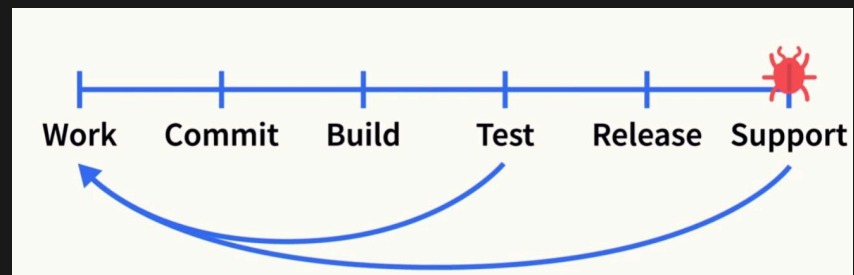
2



1



2



3

Continuous Experimentation

Cultura de aprendizaje y experimentación constante.

Práctica continua y disposición para adquirir nuevas habilidades.

When can you apply The Three Ways?

- Para crear procesos en el equipo
- Para crear estándares en el equipo
- Como parte de un estilo de management

10 Practices to Success

10. Incident Command System

Sistema estandarizado para coordinar la respuesta a incidentes, desde pequeños incidentes hasta grandes catástrofes.

9. Developers on Call

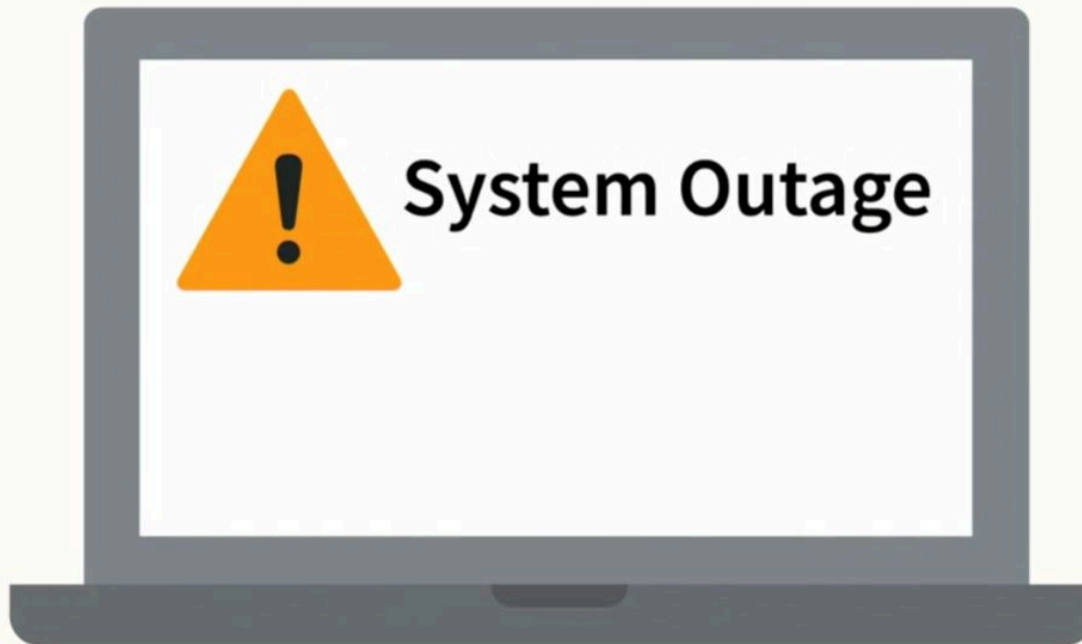
Un desarrollador es asignado para estar disponible y responder a incidentes fuera del horario laboral regular.



8. Status Pages

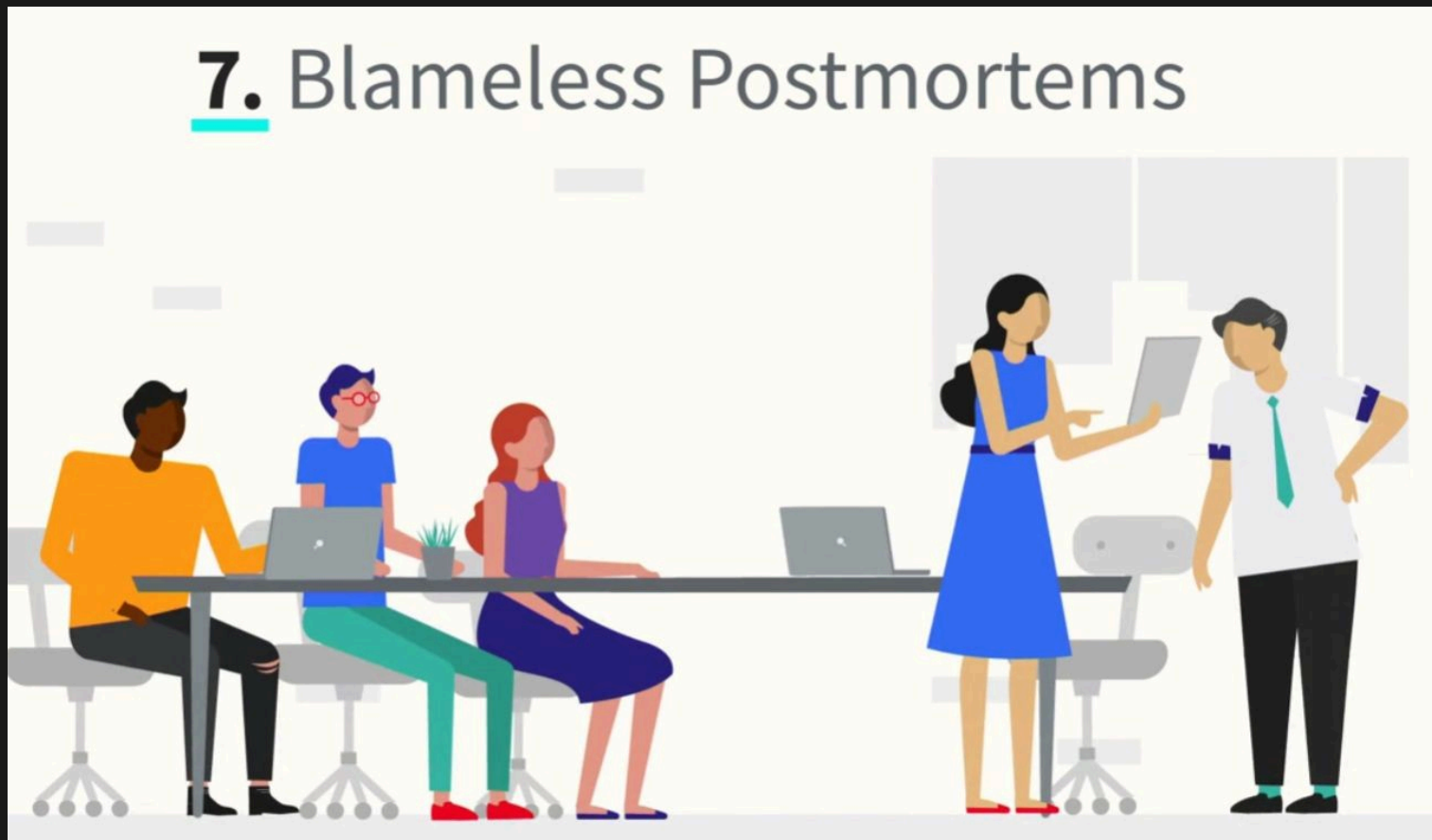
Interfaces públicas que muestran información en tiempo real sobre el estado y disponibilidad de los servicios.

8. Public Status Pages



7. Blameless Postmortems

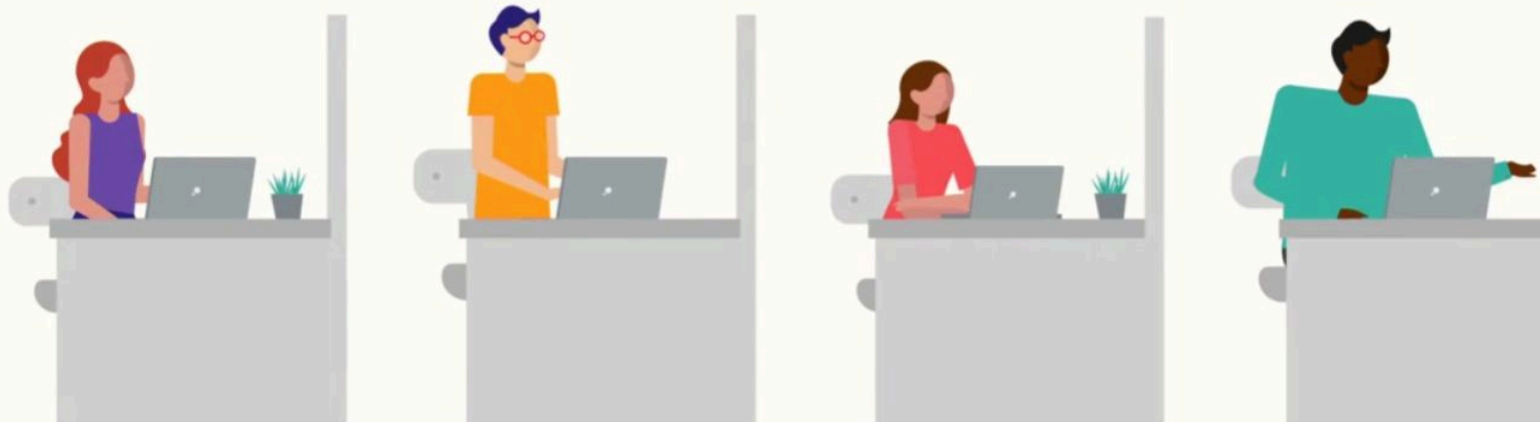
Reuniones posteriores a incidentes para analizar causas sin culpar a individuos o equipos.



6. Embedded Teams

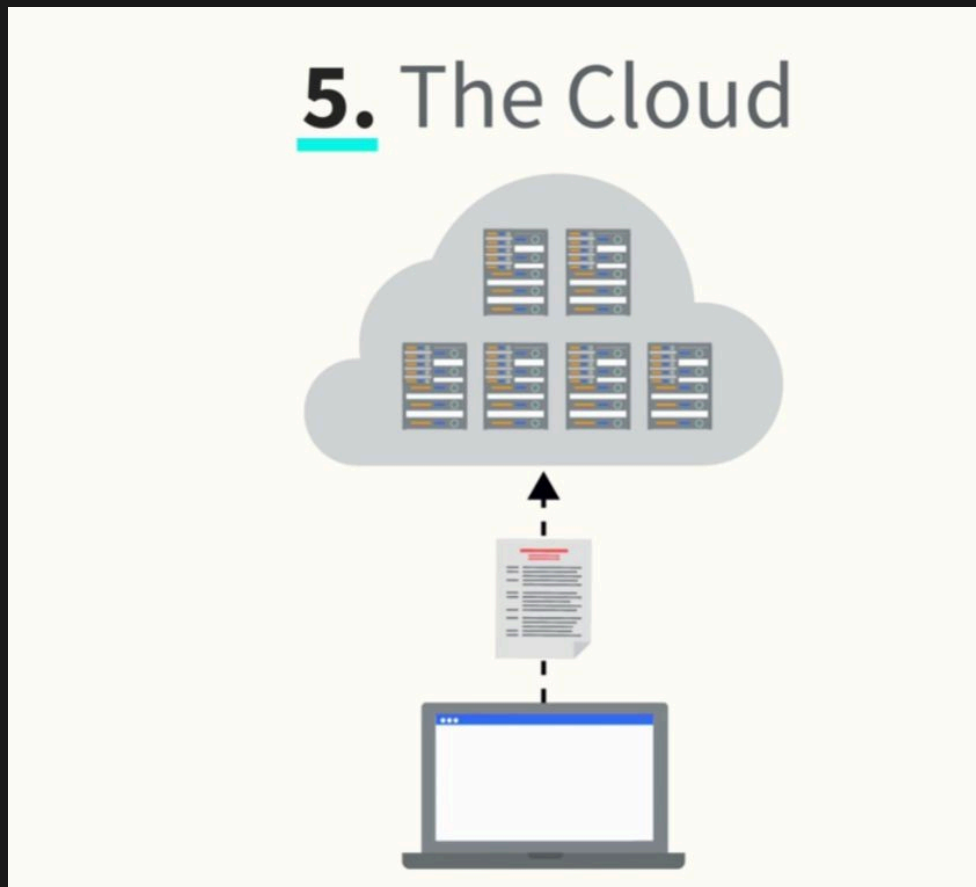
Equipos multidisciplinarios que trabajan juntos con representantes de diferentes áreas funcionales.

6. Embedded Teams



5. Cloud

La nube proporciona escalabilidad, flexibilidad y agilidad para desarrollar, desplegar y escalar aplicaciones.



4. Andon Cords

Mecanismos visuales o automáticos que alertan sobre problemas en el proceso de desarrollo, despliegue o operación.

3. Dependency Injection

Patrón de diseño de software que se centra en las dependencias débilmente acopladas.

3. Dependency Injection

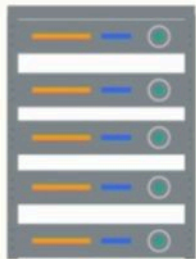


2. Blue-Green Deployment

Técnica de despliegue con dos entornos idénticos: uno activo (azul) y otro inactivo (verde).

2. Blue/Green Deployment

Load Balancer



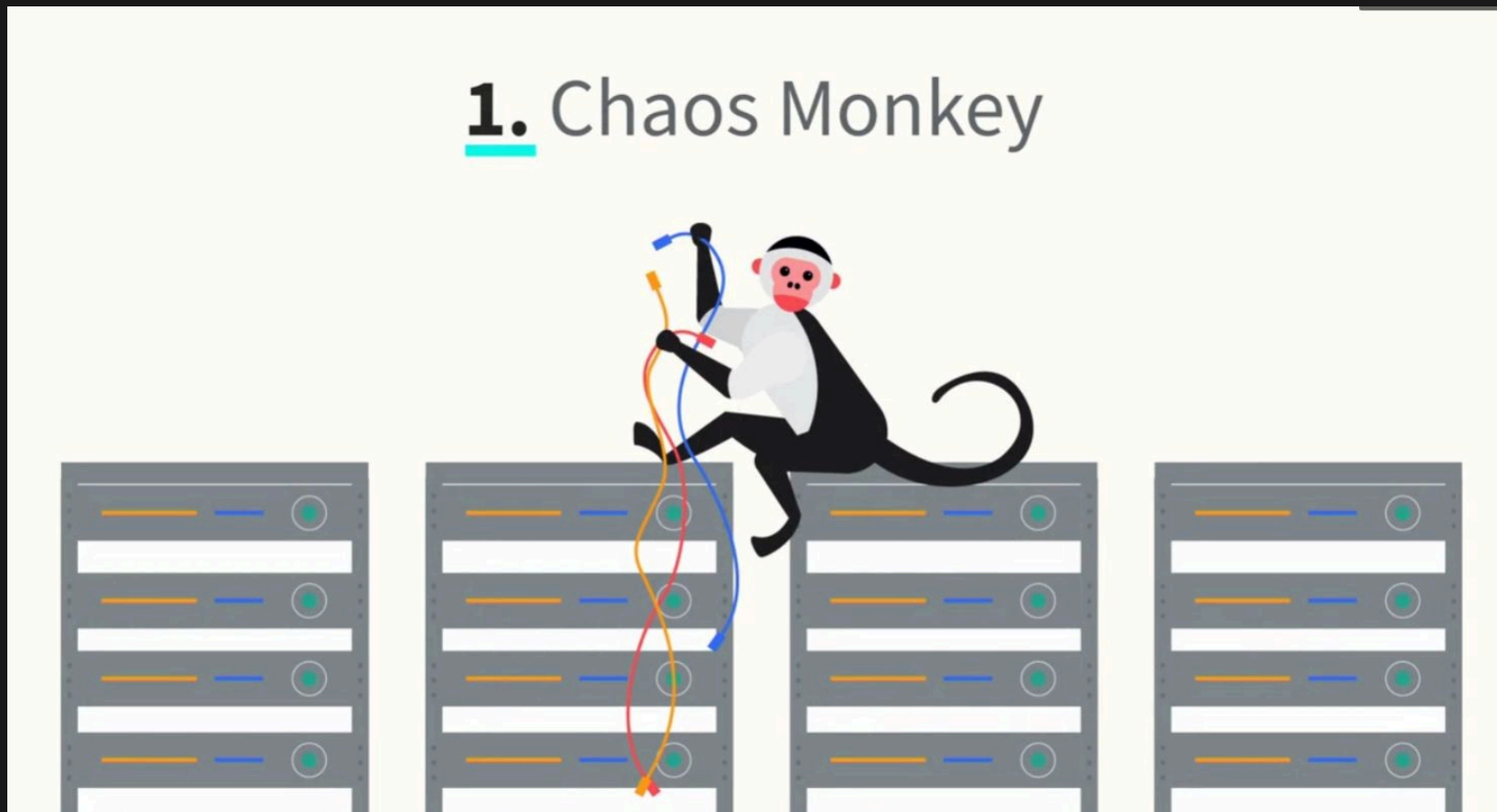
Blue



Green

1. Chaos Monkey

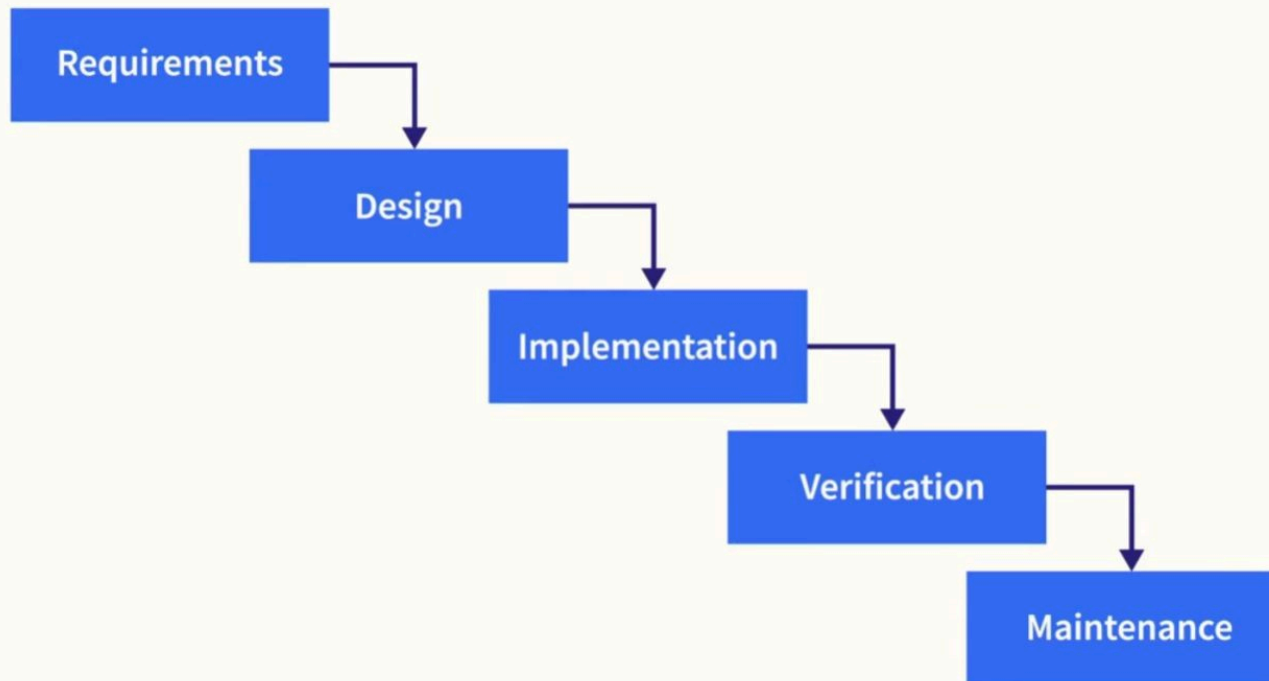
Herramienta que provoca fallos aleatorios en la infraestructura para probar la resiliencia del sistema.



Agile

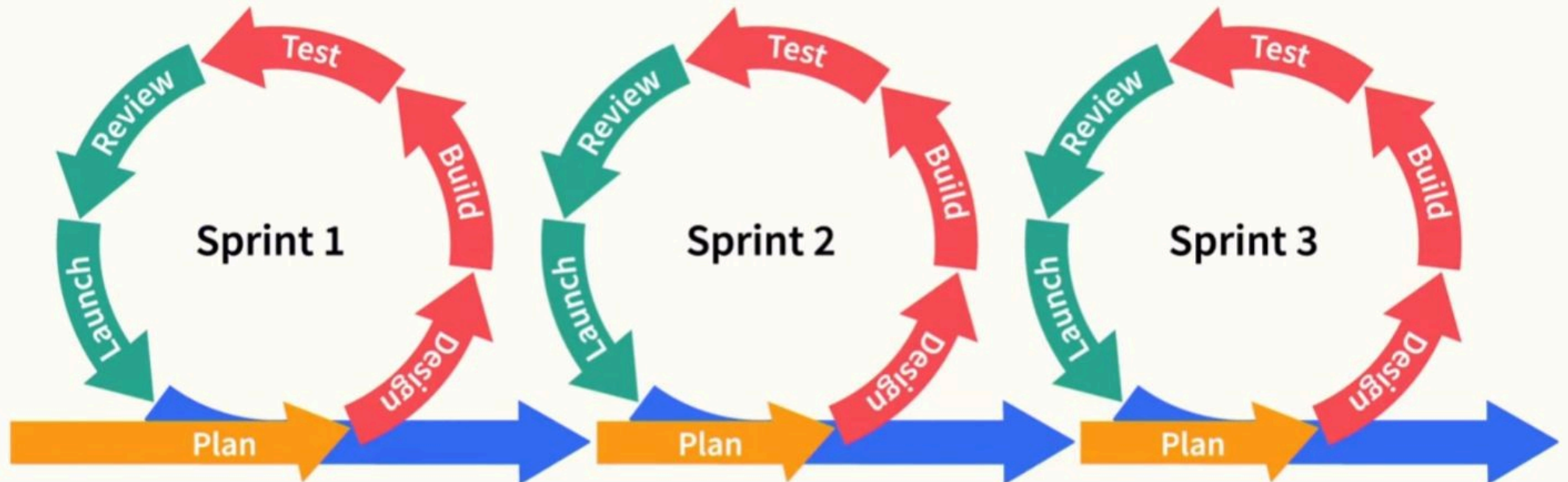
Waterfall

Enfoque tradicional de desarrollo de software con etapas secuenciales: requisitos, diseño, implementación, pruebas y mantenimiento.



Collaboration

La colaboración entre los equipos de desarrollo y los clientes es clave para adaptarse a los cambios y mejorar la calidad del producto.



Manifiesto

El Manifiesto Ágil (2001) establece valores y principios que enfatizan la colaboración, la entrega de software funcional y la respuesta al cambio.

Valores clave:

Lean

Lean Principles

Los siete principios lean del desarrollo de software:

- Eliminación de desperdicio
- Amplificación del aprendizaje
- Toma de decisiones lo más tarde posible
- Entrega rápida
- Empoderamiento del equipo
- Integridad incorporada
- Ver el todo

Build, Measure, Learn

Metodología propuesta por Eric Ries en “Lean Startup”:
construir-medir-aprender.

Entrega del producto mínimo viable → Retroalimentación →
Iteración

What is DevOps not?

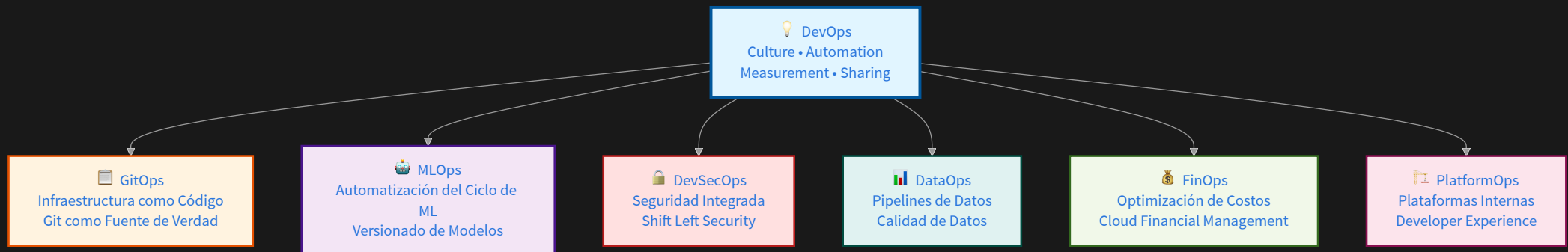
What is DevOps not?

- No es 'no' operaciones
- No es simplemente 'herramientas'
- No es simplemente 'cultura'
- No es simplemente 'Dev' y 'Ops'
- No es un título fancy
- No es todo

Prácticas Derivadas de DevOps

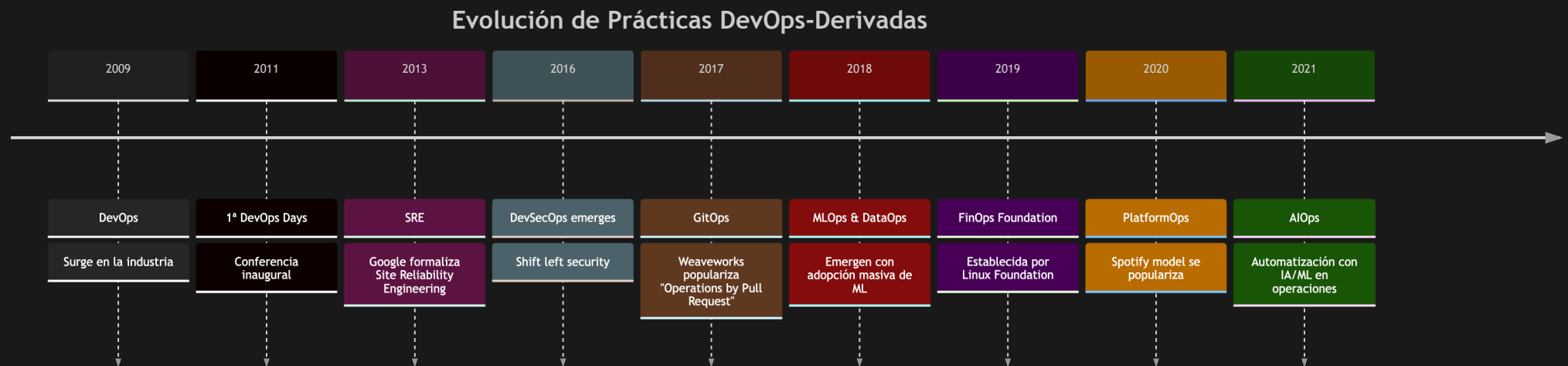
DevOps Despierta Prácticas Especializadas

El éxito de DevOps ha llevado a la evolución de prácticas especializadas que aplican sus principios fundamentales a dominios específicos.



Cronología: Evolución de Prácticas Derivadas

La evolución de DevOps a prácticas especializadas refleja cómo las organizaciones adaptan sus metodologías a nuevos desafíos.



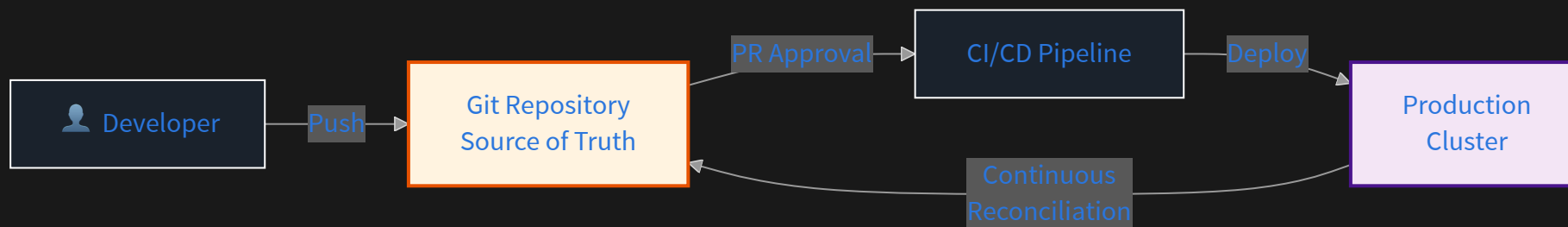
GitOps

GitOps: ¿Qué es y Origen?

GitOps es un framework operacional que aplica los mejores prácticas de desarrollo de software—control de versiones, colaboración, compliance, CI/CD—a la automatización e infraestructura.

Origen: Acuñado por Alexis Richardson de **Weaveworks** (2017) en el artículo “GitOps - Operations by Pull Request”. Representa la convergencia de Git (control de versiones) + Ops (operaciones).

Principio Central: Git es la **única fuente de verdad** para estados deseados de sistemas e infraestructura.



GitOps: Prácticas Clave

GitOps se basa en estos principios operacionales:

1. **Git como Fuente de Verdad:** Todo estado deseado vive en Git (código, configuración, infraestructura)
2. **Cambios por Pull Request:** Todos los cambios pasan por revisión colaborativa antes de producción
3. **Infraestructura Declarativa:** Descripción del estado deseado, no pasos procedimentales
4. **Reconciliación Automática:** Sistema continuamente sincroniza estado deseado (Git) con estado actual (cluster)
5. **Auditabilidad Completa:** Historial completo de cambios; trazabilidad de quién, qué, cuándo, por qué
6. **Rollback Seguro:** Revertir cambios simplemente haciendo revert en Git

Herramientas GitOps populares: ArgoCD, Flux, Weave GitOps

GitOps & Cultura DevOps

GitOps es una manifestación operativa de los pilares de DevOps:

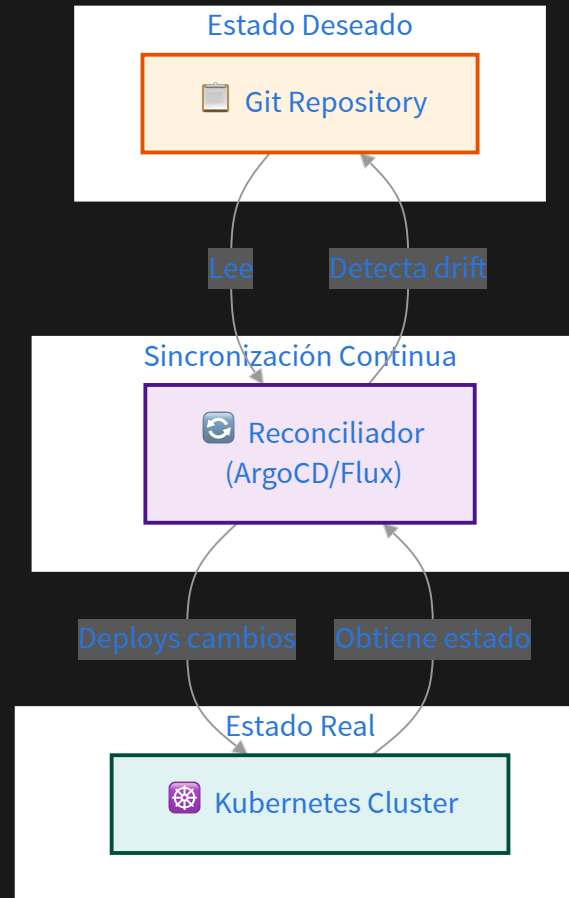
CAMS Pilar	Cómo GitOps lo Implementa
Cultura	PRs fuerzan colaboración y revisión; responsabilidad compartida en infraestructura
Automatización	Reconciliación automática y continua sincroniza estado; elimina trabajo manual
Medición	Auditoría completa de cambios; métricas de velocidad de despliegue; tracking de quién/cuándo
Compartir	PR reviews facilitan transferencia de conocimiento; historial Git es documentación viviente

Beneficios de GitOps

Beneficios GitOps:

- Velocidad: Despliegues rápidos sin cambios manuales
- Confiabilidad: Reconciliación automática previene drift
- Observabilidad: Auditoría completa de cambios
- Recuperación: Rollbacks son un simple git revert

GitOps: Reconciliación Continua



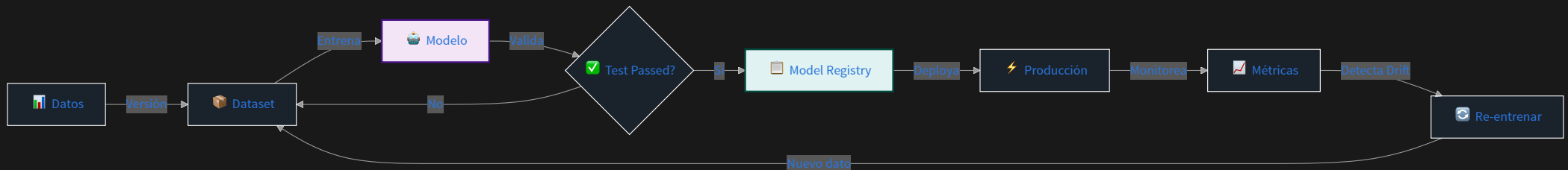
MLOps

MLOps: ¿Qué es y Origen?

MLOps (Machine Learning Operations) extiende principios de DevOps al ciclo de vida completo del machine learning: datos, entrenamiento, validación, despliegue, monitoreo y reentrenamiento.

Origen: Emergió alrededor de **2018-2019** conforme las organizaciones comenzaban a desplegar múltiples modelos en producción. Formalizado por researchers de Google, Uber, Databricks y comunidades como MLOps.community.

Desafío Único: A diferencia de software tradicional, los modelos de ML pueden degradarse (concept drift, data drift) incluso sin cambios de código.

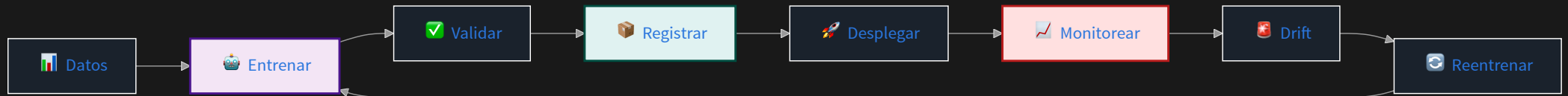


MLOps: Prácticas Clave

MLOps se articula alrededor de estas prácticas:

1. **Versionado de Datos:** Datos no son código, pero requieren versionado como código (DVC, Delta Lake)
2. **Versionado de Modelos & Hiperparámetros:** Registry centralizado de modelos (MLflow Model Registry)
3. **Pipelines Reproducibles:** Orchestración con dependencias (Airflow, Kubeflow)
4. **Validación Automatizada:** Tests para accuracy, bias, fairness, data quality
5. **Feature Store:** Repositorio centralizado y versionado de features (Tecton, Feast)
6. **Detección de Drift:** Monitorea si datos o performance del modelo degradan
7. **Automatic Retraining:** Triggers que re-entrenan si drift es detectado
8. **Model Serving:** Infraestructura de baja latencia para inferencia (KServe, Seldon, BentoML)

MLOps: Pipeline End-to-End



MLOps & Cultura DevOps

MLOps extiende los pilares de DevOps a machine learning:

CAMS Pilar	Cómo MLOps lo Implementa
Cultura	Data scientists, ML engineers, y ops colaboran; responsabilidad compartida de modelos en producción
Automatización	Pipelines automatizados (data → training → validation → deployment → retraining)
Medición	Métricas de modelo (accuracy, precision, recall), data quality, inference latency, drift detection
Compartir	Model registry compartido, experiment tracking transparente, feature store documentado

Beneficios de MLOps

Beneficios MLOps:

- **Reproducibilidad:** Mismo dato + código = mismo modelo
- **Mantenibilidad:** Modelos se reentrenan automáticamente
- **Confiabilidad:** Validación automática detecta problemas
- **Escalabilidad:** Inference y reentrenamiento optimizados

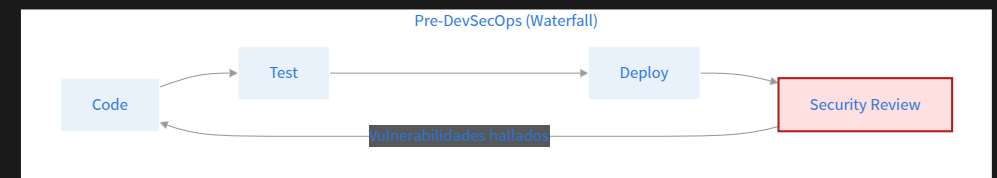
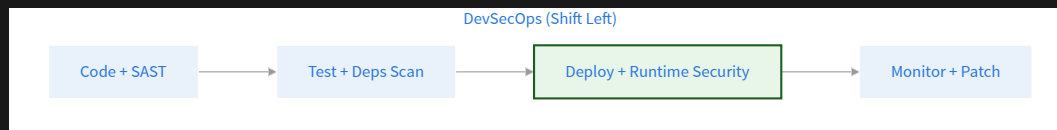
DevSecOps

DevSecOps: ¿Qué es y Origen?

DevSecOps integra seguridad en cada etapa del ciclo de vida de desarrollo y operaciones, desde diseño hasta producción. “Shift left security”—detectar vulnerabilidades tempranamente, no al final.

Origen: Emergió alrededor de **2016-2017** como respuesta a: - Cifras de brechas de seguridad en aumento - Regulaciones (GDPR, HIPAA, etc.) más estrictas - Adopción de cloud sin madurez de seguridad

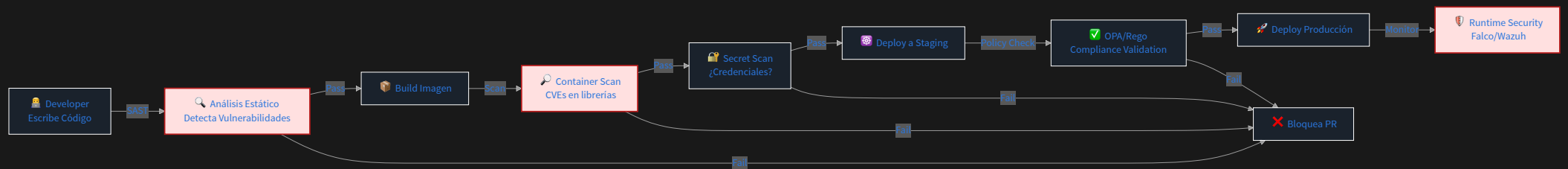
Contraste: DevOps tradicional busca velocidad. DevSecOps busca velocidad + seguridad sin trade-off—“Security at Speed”.



DevSecOps: Prácticas Clave

DevSecOps se basa en estos pilares:

DevSecOps: Pipeline de Seguridad



DevSecOps & Cultura DevOps

DevSecOps no es un gatekeeping; es colaboración integrada:

CAMS Pilar	Cómo DevSecOps lo Implementa
Cultura	Seguridad es responsabilidad de todos (dev, sec, ops); no es gatekeepers finales
Automatización	Scanning automático en each stage (SAST, deps, containers, IaC, runtime)
Medición	Métricas: MTTR de vulnerabilidades, cobertura de scanning, policy compliance
Compartir	Threat hunting insights, blameless postmortems de incidentes, training continuo

Principios DevSecOps: - **Shift Left:** Detectar problemas ANTES de que lleguen a producción - **Automated Compliance:** Políticas enforced sin fricción humana - **Speed ≠ Insecurity:** Seguridad no ralentiza; automatización lo permite - **Culture:** Security es enabling, not blocking

⚠ Vulnerability Found

Blocks PR

Auto Remediation
o Manual Fix

Retry

CI/CD Pipeline Seguro

Code Commit

Escanea

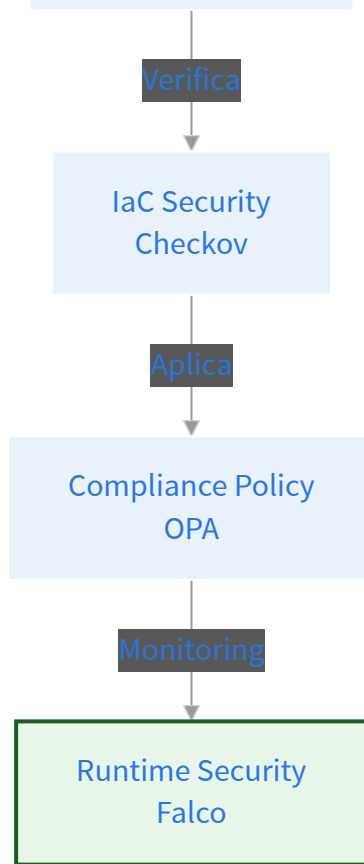
SAST
Secrets

Valida

Dependency Scan
CVEs

Chequea

Container Scan
Base Image



Otras Prácticas Derivadas de DevOps

Otras Prácticas Derivadas (1/2)

El éxito de DevOps ha inspirado prácticas especializadas en otros dominios:

Práctica	Descripción	Métrica Clave	Pilar DevOps Énfasis
DataOps	Automatización y gobierno de pipelines de datos	Data quality score, pipeline latency	Automatización
FinOps	Optimización de costos cloud como problema de ingeniería	Cost per unit of value	Medición
PlatformOps	Plataformas internas para developer self-service	Developer self-service rate	Cultura

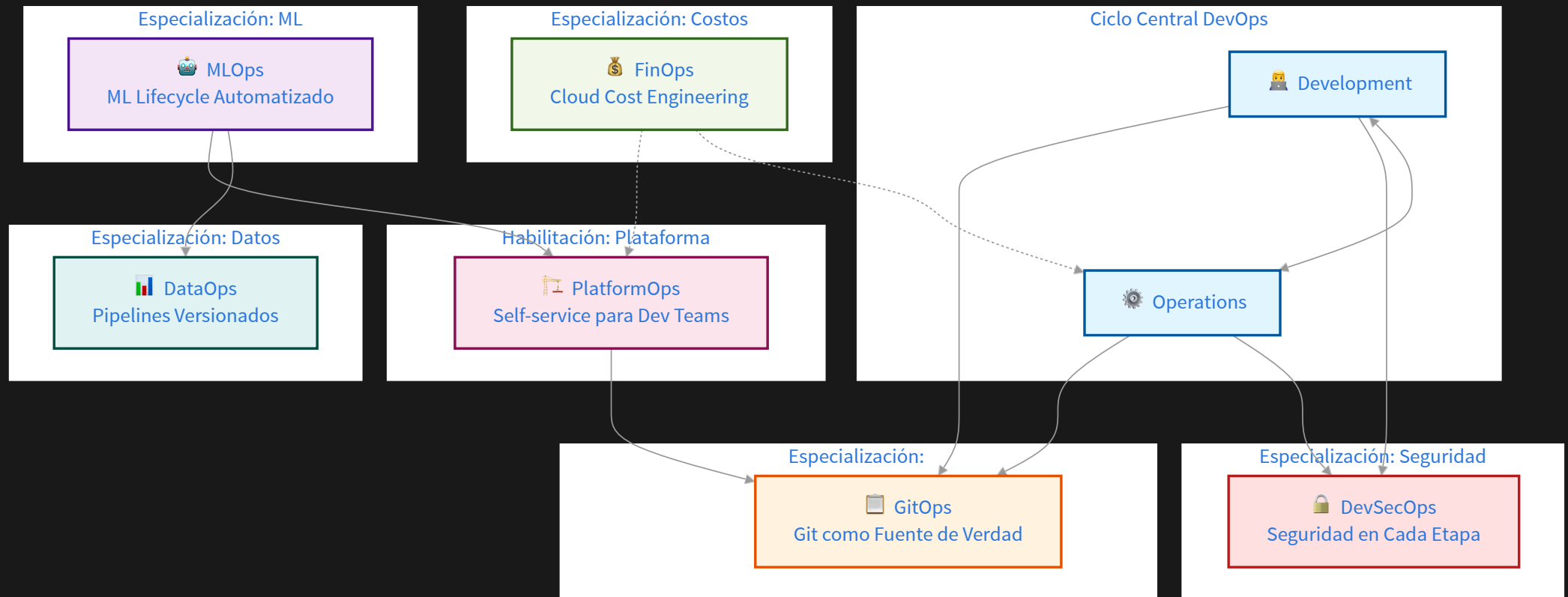
Otras Prácticas Derivadas (2/2)

Más extensiones de DevOps orientadas a confiabilidad, seguridad y operaciones inteligentes:

Práctica	Descripción	Métrica Clave	Pilar DevOps Énfasis
SRE	Confiabilidad con SLOs y error budgets	SLO compliance, error budget utilization	Medición
AIOps	IA para detección y remediación de anomalías	Incident MTTR, false positive rate	Automatización
SecOps	Operación de seguridad y	Threat detection time, incident response time	Automatización

Integración: Cómo Trabajan Juntas

Estas prácticas no son aisladas; interactúan en una organización moderna de DevOps:



Conclusión

DevOps: Un Movimiento Cultural en Evolución

DevOps no es una herramienta, no es un equipo, no es un título. Es un movimiento cultural que **evoluciona** para resolver desafíos específicos de nuevas tecnologías y dominios.

Los pilares fundamentales permanecen constantes:

- **Cultura:** Romper silos, colaborar
- **Automatización:** Eliminar trabajo manual
- **Medición:** Tomar decisiones basadas en datos
- **Compartir:** Transferir conocimiento, aprender continuamente

Prácticas como **GitOps**, **MLOps**, **DevSecOps** no son replacements; son **extensiones** que aplican estos pilares a contextos especializados.

El futuro: A medida que emergen nuevas tecnologías (quantum computing, edge computing, AI systems autónomos), probables veremos evoluciones adicionales. El principio permanece: automatización, medición, colaboración sin silos.

Bibliography

- “The Andon Cord” by John Willis
- “Blameless Postmortems and a Just Culture” by John Allspaw
- “Inversion of Control Containers and Dependency Injection Pattern” by Martin Fowler
- “DevOps Culture” by John Willis
- “GitOps - Operations by Pull Request” by Alexis Richardson (Weaveworks, 2017)
- “MLOps.community: Aspects of ML Engineering” — Resources on ML Operations
- “Shift Left Security” by Gartner
- “Site Reliability Engineering (SRE)” — Google’s SRE Book

Next

Git, Github, Docker y Kubernetes